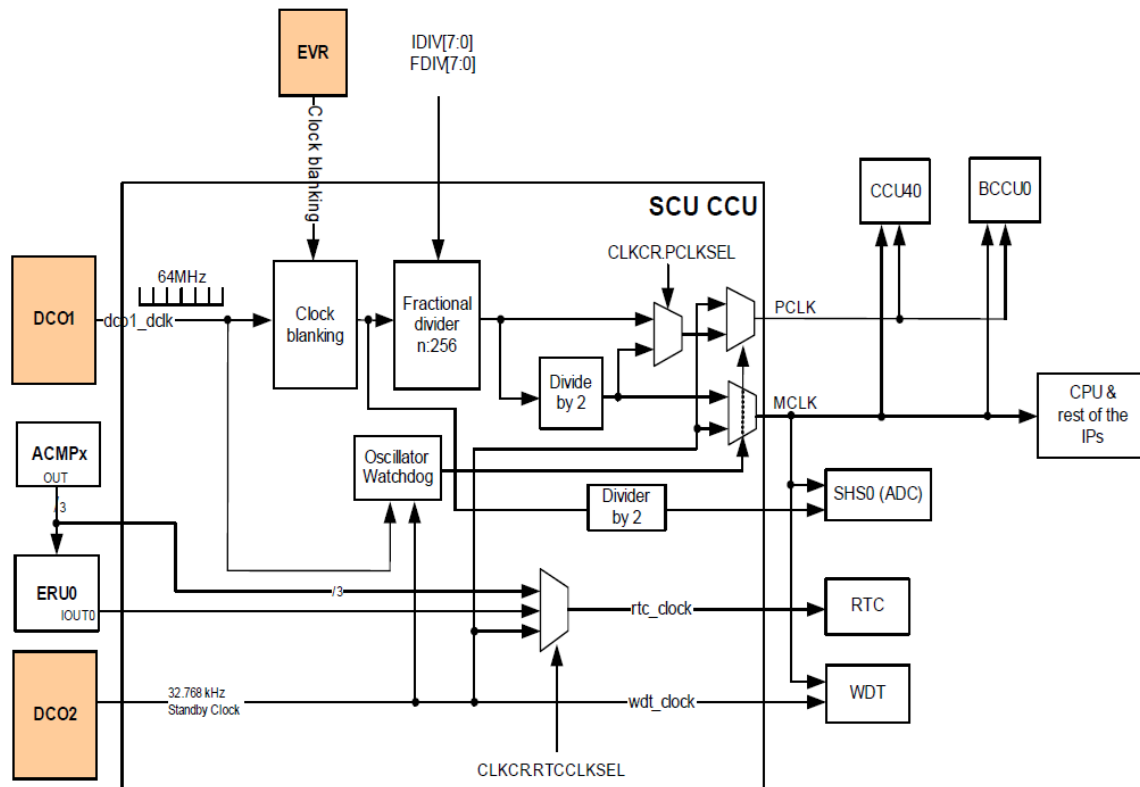


XMC1200 Boot Kit – Clocking & Interrupts

1 Einstellen der Taktfrequenz

Der XMC-Baustein ist mit einem internen Taktgenerator von 64 MHz ausgestattet. Dieser dient zur direkten Versorgung der Peripherie, während der eigentliche CPU-Kern mit maximal 32 MHz versorgt werden kann. Die SCU (Clock System Unit), die für die Bereitstellung der verschiedenen Taktsignale verantwortlich ist, lässt sich durch folgendes Blockschaltbild darstellen:



(XMC1200_Reference_Manual_v1 0_2013_03.pdf, Figure 12-4)

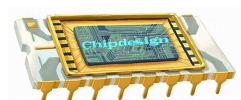
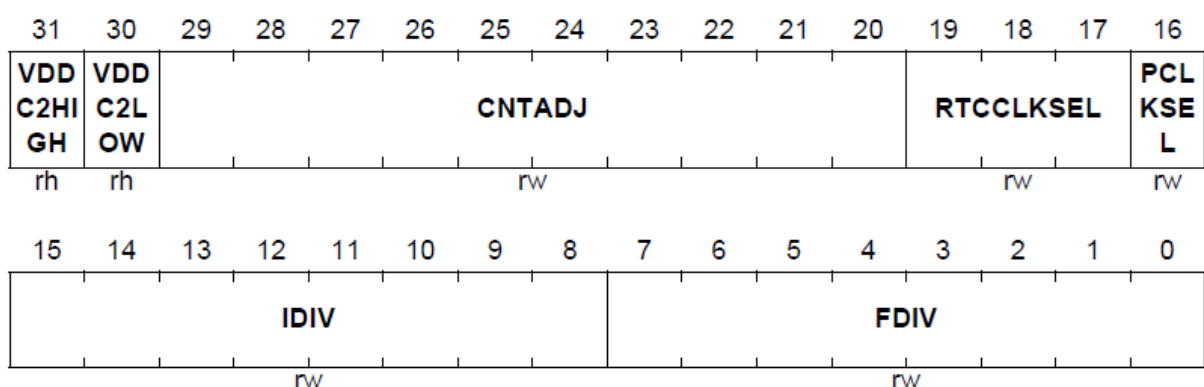
Die Taktfrequenz kann dabei im „Fractional divider“ eingestellt werden, der u.a. über den Register SCU_CLK->CLKCR (Clock Control Register) gesteuert wird:

CLKCR

Clock Control Register

(0300_H)

Reset Value: 3FF0 0400_H



Field	Bits	Type	Function
FDIV	[7:0]	rw	Fractional Divider Selection Selects the fractional divider to be $n/256$, where n is the value of FDIV and is in the range of 0 to 255. For example, writing 0001_B to FDIV selects the fractional divider to be $1/256$. This bit is protected by the bit protection scheme as described in Memory Organization chapter <i>Note: Fractional divider has no effect if IDIV = 00_H.</i>
IDIV	[15:8]	rw	Divider Selection 00_H Divider is bypassed. 01_H 1; MCLK = 32 MHz 02_H 2; MCLK = 16 MHz 03_H 3; MCLK = 10.67 MHz 04_H 4; MCLK = 8 MHz FE_H 254; MCLK = 126 kHz FF_H 255; MCLK = 125.5 kHz This bit is protected by the bit protection scheme as described in Memory Organization chapter

Die CPU-Frequenz lässt sich für $IDIV > 0$ nach folgender Formel berechnen:

$$MCLK = \frac{32 \text{ MHz}}{IDIV + FDIV/256}$$

Für $IDIV=0$ wird der Teiler umgangen, so dass der Prozessor mit 32 MHz arbeitet. Die Default-Werte sind $FDIV=0$ und $IDIV=4$, d.h. die CPU läuft Default-mäßig mit 8 MHz.

Die Bits von FDIV und IDIV sind gegen versehentliches Beschreiben geschützt. Vor dem Ändern muss daher das Beschreiben über das Register SCU_GENERAL->PASSWD geöffnet werden:

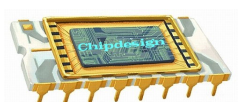
```
//----CLOCK-SETUP-----
SCU_GENERAL->PASSWD = 0xC0; // Opens access to writing of all protected bits

... // Modify protected bits

SCU_GENERAL->PASSWD = 0xC3; // Closes access to writing of all protected bits
//-----
```

Erzeugen Sie ein neues Projekt „Exercise_3_clockSpeed“ analog zum Übungsblatt 1, und übernehmen Sie das File „Main.c“ aus dem Lauflichtbeispiel aus Übung 2.

Experimentieren Sie mit der Takteinstellung, und lassen Sie Ihr Lauflicht neben der Default-Frequenz von 8 MHz mit den Frequenzen 32 MHz, 1 MHz und 125.5 kHz laufen.



2 Berechnung der Laufzeit

Die Laufzeit im obigen Programm hängt praktisch nur von der Scheife wait() ab. Achten Sie darauf, dass die Schleife im Quellcode folgendermaßen realisiert ist:

```
void wait (void)
{ for (register int i = 0; i <= 100000; i++ ) {} }
```

Untersuchen Sie die entsprechenden Assembler-Befehle, die die innere Schleife zum Zählen ausmacht. Das technische Referenzmanual des ARM Cortex-M0 Prozessors beinhaltet die Anzahl der benötigten Maschinenzyklen pro Befehl. Hier ein Auszug (Cortex-M0 Technical Reference Manual, Copyright 2009 ARM Limited, Abschnitt 3.3):

Operation	Assembler	Cycles
Move	mov Rd, #<imm>	1
	mov Rd, Rm	1
Add	add Rd, #<imm>	1
	add Rd, Rm	1
Load register	ldr Rd, [Rn, #<imm>]	2
Store register	str Rd, [Rn, #<imm>]	2
Compare	cmp Rn, Rm	1
Branch conditional	B<cc> <label>	3 if taken, 1 if not taken
Branch unconditional	B <label>	3
Push	push <reglist>	1+N ^a
	push {<reglist, lr>}	1+N ^a
Pop	pop {<reglist>}	1+N ^a
	pop {<reglist, pc>}	4+N ^a

a) N is the number of elements

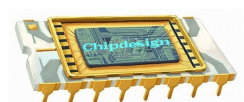
Dabei sind bei der XMC 1200 Implementierung für einen Maschinenzyklus sind zwei Takte notwendig. Welche Zeit braucht der Prozessor in etwa, um die Scheife wait() bei einer Taktfrequenz von 1 MHz einmal zu durchlaufen? Führen Sie mit verschiedenen Frequenzen Experimente durch, und versuchen Sie, aus der Geschwindigkeit des Lauflichts die Taktfrequenz abzuschätzen.

3 Verwendung der Interrupt-Funktion

Erzeugen Sie ein neues Projekt „Exercise_3_flash“, und übernehmen Sie das File „Main.c“ aus dem Beispiel „Exercise_2_pattern“ aus Übung 2.

Der Prozessor besitzt einen 24-Bit Zähler (SysTick), der mit jedem Takt jeweils um 1 verringert wird. Wenn der Zähler Null erreicht, kann der Prozessor einen Interrupt auslösen, und den Zähler wird mit einem beliebigen neuen Startwert laden. Auf diese Weise lassen sich in regelmäßigen Abständen Programme ausführen, ohne den Prozessor zu belasten bzw. ohne Warteschleifen wie unser „wait()“ zu verwenden. Um den Timer zu verwenden, müssen folgende 3 Register beschrieben werden (siehe auch Reference-Manual Abschnitt 2.9.2):

```
SysTick->LOAD = count; // Set SysTick counter to given value
SysTick->VAL   = 0x00;  // Start counter with 0
SysTick->CTRL  = 0x07;  // 0x01: Enable SysTick counter
                  // 0x02: Enable interrupt request on counter 0
                  // 0x04: Use processor clock as clock source
```



Dabei gibt die Variable „count“ den Wert an, mit dem der Zähler bei jedem Interrupt neu geladen wird. Über SysTick->CTRL wird der Interrupt gestartet. Um bei einem 8-MHz Takt alle 0,5 s eine Unterbrechung zu erzeugen, muss SysTick->LOAD somit mit $8.000.000/2$ geladen werden. Der Anwender muss schließlich eine Funktion mit dem hart-kodierten Namen „SysTick_Handler“ einfügen, die dann jeweils alle 0,5 s aufgerufen wird:

```
void SysTick_Handler (void)
{ // Insert interrupt code here }
```

Aufgabe:

Ändern Sie den Code derart ab, dass mit Hilfe der SysTick-Funktionalität alle LEDs im Sekundentakt blinken. Gerne können Sie dazu die Funktion writeLED() verwenden.

Damit der Prozessor zwischen den Interrupt-Aufrufen keine unnötige Energie verbraucht, sollte er entsprechend „schlafen“ gelegt werden. Dies kann z.B. mit dem Assembler-Befehl WFI (Wait for Interrupt) gesehen. Der Prozessor hält dann so lange an, bis der nächste Interrupt auftritt. Im C-Code lässt sich dieser Assembler-Befehl direkt einbetten:

```
__ASM("wfi");
```

Verwenden Sie diesen Befehl, um den Energieverbrauch zwischen den Interrupt-Aufrufen zu minimieren, und zeigen Sie das Ergebnis dem Betreuer!

4 Lauflicht mit Interrupt (Tricky)

In dieser Übung sollen Sie in einem neuen Projekt „Exercise_3_runningLightInt“, ein Lauflicht aus Übungsblatt 2 programmieren, das anstelle der Funktion wait() die Interruptsteuerung verwendet.

Um die Einstellungen einfacher zu gestalten, werden von der Entwicklungsumgebung die vordefinierten Funktionen SystemCoreClockUpdate und SysTick_Config zur Verfügung gestellt. „SystemCoreClockUpdate“ liest dabei die aktuelle Taktfrequenz, und stellt sie in der Variable „SystemCoreClock“ zur Verfügung (in Hz). SysTick_Config startet den SysTick-Interrupt mit entsprechendem Zählwert:

```
//-----Start SysTick interrupt-----
SystemCoreClockUpdate();
SysTick_Config(SystemCoreClock / 5); // used for 5 times per second
//-----
```

Verwenden Sie beide Funktionen, und erzeugen Sie das Lauflicht mit der Interrupt-Funktionalität. Experimentieren Sie auch mit verschiedenen Taktfrequenzen, und untersuchen Sie den Einfluss des Taktes auf die Geschwindigkeit des Lauflichts.

